TRACEFORGE

Incident Reconstruction and IOC Correlation Workbench

PROJECT REPORT

Submitted by

Harsh Garate

Abstract

TraceForge is a local incident reconstruction and IOC (Indicator of Compromise) correlation workbench developed to support the analysis of security log files collected from different sources. The application imports supported log formats, converts records into a common event structure, normalises timestamps, records evidence hashes and reports data-quality issues. Its purpose is to reduce the manual effort involved in reviewing separate log files and to provide a structured workflow for investigating potentially suspicious activity.

The system applies built-in detection rules and supports a constrained subset of imported Sigma rules. Findings are correlated using shared actor entities and configured time windows to create incidents. Analysts can review event timelines, inspect linked evidence, add notes and indicators, record verdicts and export investigation reports. The project includes a browser-based local console, a command-line workflow, SQLite case persistence and export options including PDF, Markdown, JSON, CSV, STIX 2.1 and MITRE ATT&CK Navigator layers.

TraceForge is designed for local, single-analyst use and processes user-supplied files; it is not a live monitoring platform and does not automatically confirm that an attack has occurred. The project archive includes a synthetic demonstration involving 555 events from four sources, 17 detection findings and four correlated incidents. In the latest test execution captured during report revision, 175 tests passed and two tests failed, with 71 warnings. The failures were test_nested_json_and_ndjson (an expected username did not match the anonymised fixture value) and test_clear_marking_removes_identifiers_from_every_report_format (the test detected an identifier-like value in an exported report). These results are a diagnostic snapshot, not a successful final test run.

Keywords: Python, cybersecurity, incident reconstruction, log analysis, IOC correlation, Sigma, MITRE ATT&CK, evidence integrity, SOC.

Chapter

CONTENTS

1 INTRODUCTION

1.1 Introduction

1.2 Background

1.3 Problem Statement

1.4 Objectives

1.5 Purpose, Scope and Applicability

1.6 Achievements

1.7 Organisation of Report

2 LITERATURE SURVEY

2.1 Security Log Analysis

2.2 Incident Response

2.3 MITRE ATT&CK Framework

2.4 Sigma Detection Rules

2.5 Evidence Integrity and Auditability

2.6 Summary of Literature Survey

3 REQUIREMENTS AND ANALYSIS

3.1 Problem Definition

3.2 Requirements Specification

3.3 Hardware and Software Requirements

3.4 Preliminary Product Description

3.5 Conceptual Model

4 SYSTEM DESIGN

4.1 System Architecture

4.2 Basic Modules

4.3 Data Design

4.4 Procedural Design

4.5 Detection and Correlation Design

4.6 Security Considerations

4.7 Test Case Design
5 IMPLEMENTATION AND TESTING
5.1 Implementation Approach
5.2 Log Ingestion and Normalisation
5.3 Detection and Correlation Implementation
5.4 Security Implementation
5.5 Testing Approach
5.6 Challenges and Improvements
6 RESULTS AND OUTPUT
6.1 Final Output
6.2 Analyst Workflow
6.3 Project Screenshots
6.4 Results Summary
6.5 Limitations
7 FUTURE SCOPE
8 CONCLUSION
BIBLIOGRAPHY

List of Figures

Figure 3.1: System Context Diagram

Figure 4.1: System Architecture of TraceForge

Figure 4.2: Project Data Flow Diagram

Figure 4.3: Incident Correlation Workflow

Figure 6.1: TraceForge Dashboard

Figure 6.2: Evidence Import and Data Quality

Figure 6.3: Detection Findings and Incident Timeline

Figure 6.4: IOC Management

Figure 6.5: Reports & Investigation Summary

Figure 7.1: Recommended Future Development Roadmap

List of Tables

Table 2.1: Summary of Literature Survey

Table 3.1: Functional Requirements

Table 3.2: Non-Functional Requirements

Table 3.3: Hardware Requirements

Table 3.4: Software Components and Tools

Table 4.1: Core Modules and Responsibilities

Table 4.2: Test Case Design

Table 5.1: Automated Test Result

Table 6.1: Results Summary

Chapter 1

INTRODUCTION

1.1 Introduction

Cybersecurity investigations often require analysts to examine records produced by different systems, including authentication services, endpoints, web servers and network devices. These sources may use different field names, timestamp formats and event structures. When the records are stored in separate files, identifying relationships between events and reconstructing the sequence of suspicious activity can become time-consuming.

TraceForge is a local incident reconstruction and IOC correlation workbench developed to bring supported security log files into one analysis workspace. It normalises records into a common event model, evaluates them using detection rules, links related findings and presents incidents with supporting evidence. The analyst can review the evidence, record a verdict, add notes and indicators, and export a structured report.

1.2 Background

Security log analysis is an important activity in Security Operations Centre (SOC) workflows. Analysts review event records to identify unusual authentication attempts, reconnaissance, suspicious process activity, possible persistence and other behaviours that may require investigation. A common event structure makes it easier to compare records from different sources.

TraceForge applies this concept to user-provided files. It includes parsers, timestamp normalisation, data-quality reporting, rule-based detection, incident correlation and evidence-linked reporting. The application is intended as an offline investigation aid and does not collect telemetry continuously from a live network.

1.3 Problem Statement

When security logs are distributed across different files and formats, analysts must manually parse and compare timestamps, IP addresses, user accounts, hosts and event descriptions. This can lead to slow investigations, missed relationships, inconsistent notes and reports that are difficult to reproduce. The project addresses the need for a local tool that can ingest supported logs, organise events, highlight suspicious patterns, correlate related findings and preserve evidence references in a structured investigation workflow.

1.4 Objectives

- Import supported CSV, JSON, NDJSON and raw text logs, along with selected Windows Event XML, Sysmon, Zeek and CloudTrail formats.
- Convert source-specific fields into a canonical event schema and normalise timestamps to UTC.
- Record SHA-256 hashes for evidence files and expose parsing and data-quality information.

- Detect potentially suspicious activity using built-in rules and supported imported Sigma rules.
- Correlate related findings into incidents using shared actor entities and time windows.
- Provide analyst workflows for timelines, evidence review, verdicts, notes, IOCs and response checklists.
- Generate human-readable and machine-readable investigation outputs.
- Apply local application security controls and preserve an auditable record of case changes.

1.5 Purpose, Scope and Applicability

1.5.1 Purpose

The purpose of TraceForge is to demonstrate how log ingestion, detection engineering, event correlation and evidence management can be combined into a practical local investigation tool. It aims to help an analyst move from separate log files to a reviewable incident timeline with linked evidence.

1.5.2 Scope

The current project covers local file ingestion, event normalisation, rule-based detection, IOC matching, incident correlation, analyst triage, SQLite case persistence, evidence hashing and report export. It includes a browser-based console and command-line analysis workflow.

1.5.3 Applicability

TraceForge is suitable for educational demonstrations, cybersecurity portfolios and controlled analysis of exported security logs. It may serve as a starting point for a larger case-management tool, but it is not intended to replace an enterprise SIEM, EDR platform or incident-response team.

1.6 Achievements

- Implemented multi-format log ingestion and canonical event normalisation.
- Added built-in detection rules and constrained Sigma rule support.
- Implemented correlation of related findings into incidents.
- Added evidence hashing and integrity verification.
- Provided analyst review functions including timelines, notes, verdicts and IOCs.
- Added report and data exports, including selected security-standard formats.
- Executed the automated test suite during report revision: 175 tests passed, 2 failed and 71 warnings were reported. The two failures remain under investigation; the suite is not yet fully passing.

1.7 Organisation of Report

This report is organised into eight chapters. Chapter 1 introduces the project, problem, objectives and scope. Chapter 2 presents the literature survey. Chapter 3 describes requirements and analysis. Chapter 4 explains system design and architecture. Chapter 5 discusses implementation and testing. Chapter 6 presents results, screenshots and limitations. Chapter 7 describes future enhancements, and Chapter 8 concludes the report.

Chapter 2

LITERATURE SURVEY

2.1 Security Log Analysis

Security log analysis involves reviewing records generated by systems, applications and network services to identify events that may indicate misuse or malicious behaviour. Different products often represent similar information using different field names and formats. A normalisation stage helps place these records into a consistent structure before searching or correlation.

TraceForge uses a canonical event model to make selected fields comparable across supported sources. The quality of the analysis still depends on the completeness and accuracy of the original logs.

2.2 Incident Response

Incident response involves preparing for, detecting, analysing, containing and recovering from cybersecurity incidents. A reliable investigation record should preserve timestamps, affected entities, evidence references, findings, decisions and recommended actions. TraceForge focuses on the analysis and documentation portion of this workflow; it does not execute containment or recovery actions.

2.3 MITRE ATT&CK Framework

MITRE ATT&CK is a knowledge base of adversary tactics and techniques based on observed behaviours. TraceForge associates supported detections with ATT&CK technique identifiers where applicable. This mapping helps describe the behaviour represented by a finding, but it does not prove that a specific actor or complete attack chain was present.

2.4 Sigma Detection Rules

Sigma is a generic format for describing log-based detection logic. TraceForge supports a constrained subset of Sigma rules and applies validation to rule content. Unsupported constructs are not treated as successful detections; they are skipped with an explanation. This allows rule-based detection to be extended while keeping the supported behaviour explicit.

2.5 Evidence Integrity and Auditability

Cryptographic hashes such as SHA-256 can be used to identify files and detect changes to their contents. TraceForge stores evidence references and hashes, and can verify whether a retained evidence file still matches its recorded digest. The audit log uses hash chaining to make changes to recorded actions detectable. A hash chain alone cannot prove that the newest records have not been removed, so the audit data directory must also be protected.

2.6 Summary of Literature Survey

Table 2.1: Summary of Literature Survey

Reference / Technology	Main Contribution	Use in TraceForge
Security log analysis	Normalisation and correlation of event records.	Canonical event model and multi-source analysis.
Incident response guidance	Structured investigation and evidence documentation.	Incident workflow, triage and reporting.
MITRE ATT&CK	Common language for adversary tactics and techniques.	Technique mapping for supported detections.
Sigma	Portable format for log-based detection rules.	Constrained imported rule support.
SHA-256	Cryptographic digest for integrity checking.	Evidence file hashing and verification.
SQLite	Embedded relational data storage.	Local case and analyst-workflow persistence.

Chapter 3

REQUIREMENTS AND ANALYSIS

3.1 Problem Definition

The project must provide a repeatable workflow for importing logs, normalising records, identifying suspicious patterns and correlating findings. The tool must also retain evidence references and allow an analyst to review the resulting incidents. It must operate locally and clearly communicate that detections are investigative leads rather than confirmed attacks.

3.2 Requirements Specification

Table 3.1: Functional Requirements

ID	Functional Requirement	Priority
FR01	Import supported log files and report parsing outcomes.	High
FR02	Normalise recognised fields and timestamps into a common event schema.	High
FR03	Calculate and retain SHA-256 evidence hashes.	High
FR04	Run built-in detection rules against normalised events.	High
FR05	Load and evaluate supported Sigma rules.	Medium
FR06	Correlate related findings into incidents.	High
FR07	Display event timelines and linked evidence.	High
FR08	Allow analyst notes, verdicts, IOCs and response checklist updates.	Medium
FR09	Persist case state locally using SQLite.	High
FR10	Export reports and selected structured formats.	High

Table 3.2: Non-Functional Requirements

Category	Requirement
Usability	The interface should present events, findings and incidents in a clear investigation workflow.
Security	Untrusted input must be validated and sensitive case data should remain local by default.
Reliability	Malformed input and unsupported formats should produce clear errors rather than silent corruption.
Maintainability	The application should separate ingestion, detection, correlation, storage and presentation logic.
Portability	The project should run in a supported Python environment with declared dependencies.

Auditability	Important case changes and evidence-integrity information should be reviewable.

3.3 Hardware and Software Requirements

Table 3.3: Hardware Requirements

Component	Recommended / project environment	Purpose
Processor	Modern dual-core or better	Runs parsing, correlation and the local application.
RAM	4 GB minimum recommended	Supports Python runtime and browser interface.
Storage	Sufficient space for source code, evidence and case database	Stores imported evidence and reports.
Display	1366 × 768 or higher	Displays the local web console.
Input	Keyboard and mouse	Used for file selection and analyst triage.

Table 3.4: Software Components and Tools

Software / Tool	Purpose
Python 3.11+	Main programming language.
FastAPI / Uvicorn	Local API and application server.
HTML, CSS, JavaScript, SVG	Browser-based interface and visual elements.
SQLite	Local case persistence.
Pandas / NumPy	Tabular data processing.
ReportLab / Matplotlib	PDF reports and visual summaries.
PyYAML	Supported Sigma rule parsing.
Cryptography	Encrypted case-file export.
Pytest	Automated testing.
Visual Studio Code / Git	Development and version control.

3.4 Preliminary Product Description

Input: TraceForge accepts analyst-selected log files in supported formats, imported Sigma rules and analyst-entered case information. Processing: the application parses and normalises records, records evidence hashes, evaluates detection rules, correlates findings and stores case state. Output: the application displays event data, findings, incidents, timelines, evidence status and analyst notes, and generates reports and selected structured exports.

3.5 Conceptual Model

The conceptual model treats TraceForge as a local analysis pipeline. User-supplied files enter the ingestion layer, where parsers convert records into canonical events. The detection engine produces findings, and the correlation engine groups related findings into incidents. The analyst reviews the resulting evidence and case context before exporting a report.

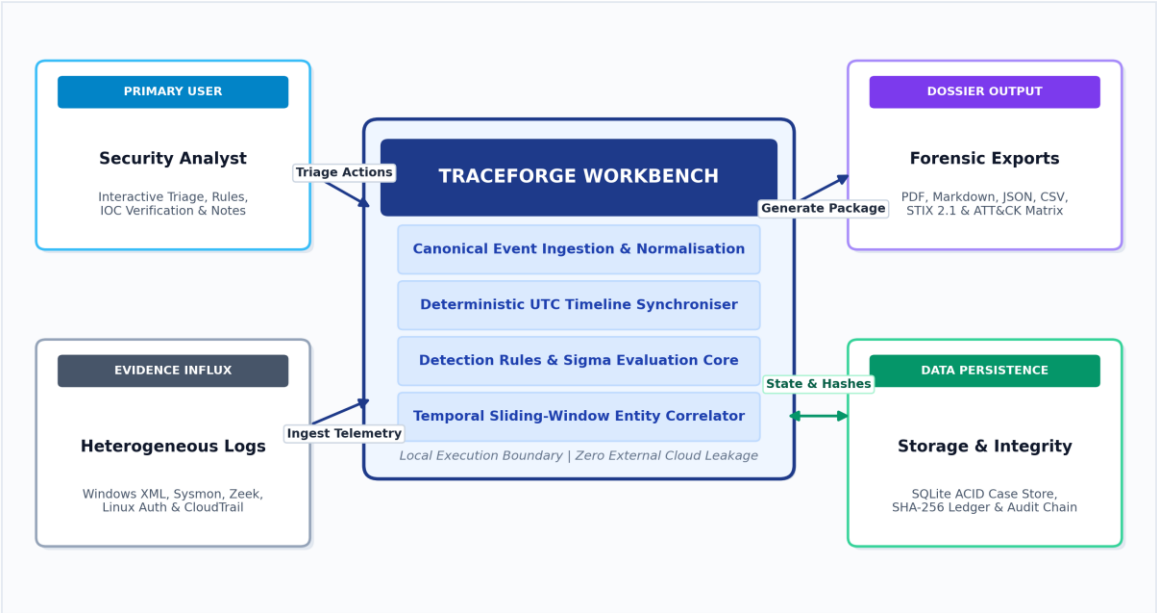


Figure 3.1: System Context Diagram

Chapter 4

SYSTEM DESIGN

4.1 System Architecture

TraceForge follows a layered local architecture. The ingestion layer reads supported input files and maps source fields into a canonical event structure. Evidence handling calculates file hashes and retains evidence references. The detection engine evaluates events against built-in and supported imported rules. The correlation module groups related findings into incidents. The service and API layers provide case data to the browser console, while the reporting layer generates exports.

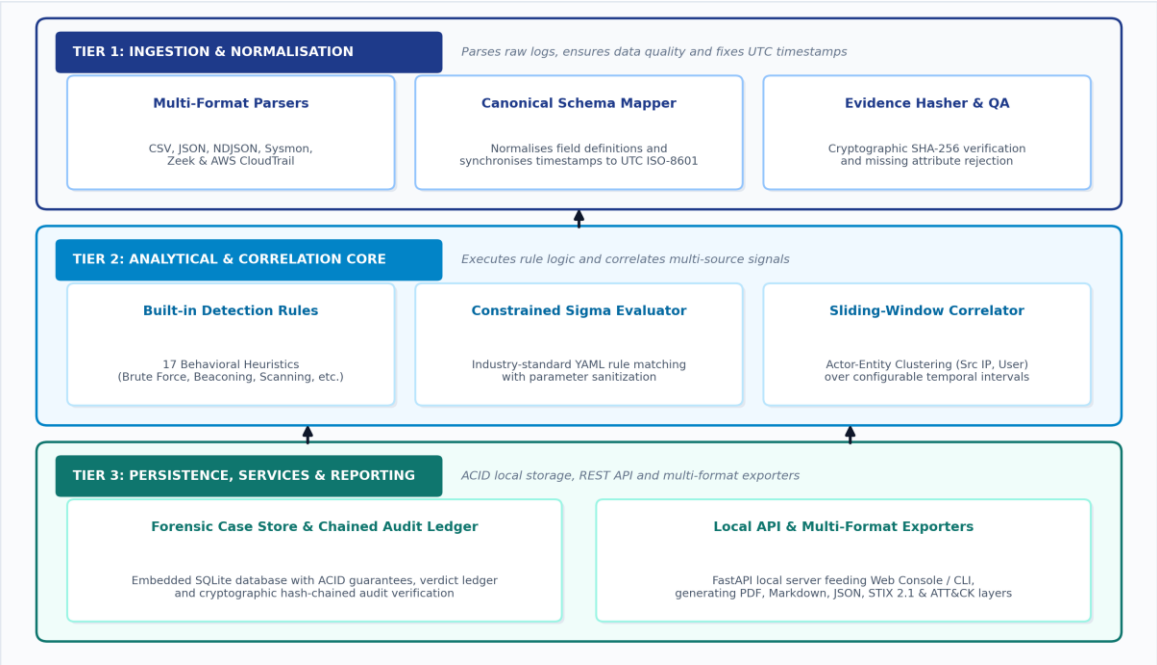


Figure 4.1: System Architecture of TraceForge

4.2 Basic Modules

Table 4.1: Core Modules and Responsibilities

Module / Component	Responsibility
Ingestion and format parsers	Read supported formats, map fields, normalise timestamps and report data quality.
Evidence handling	Create evidence copies/references and verify SHA-256 integrity.
Detection engine	Evaluate built-in detection rules and supported imported Sigma rules.
Correlation engine	Link related findings using shared actor entities and configured time windows.
Case storage and audit	Persist case state, notes, verdicts, IOCs and audit records in SQLite.
Service and API	Coordinate analysis workflows and expose application data

	to the interface.
Web console	Present dashboard, findings, incidents, timelines, IOCs and reports.
Reporting and exporters	Generate PDF, Markdown, JSON, CSV and selected security-format exports.

4.3 Data Design

4.3.1 Project Data Organisation

The project separates application source code, web interface assets, documentation, tests, configuration and runtime case data. The runtime data layer contains local case information and evidence-related records. Exact paths should be confirmed against the final submitted repository version.

4.3.2 Event and Case Data

The canonical event structure provides common fields for event time, source, event category, actor and target entities, network indicators and raw event context. Findings retain references to the events that triggered them. Incidents group related findings and provide analyst-facing context such as severity, confidence factors, state, notes and linked indicators.

4.4 Procedural Design

4.4.1 Data Flow

The main data flow begins when the analyst selects log files. The ingestion layer parses and normalises records, then stores event data and evidence metadata. The detection engine evaluates events and creates findings. The correlation engine groups related findings into incidents. The analyst reviews the incidents and exports a report.

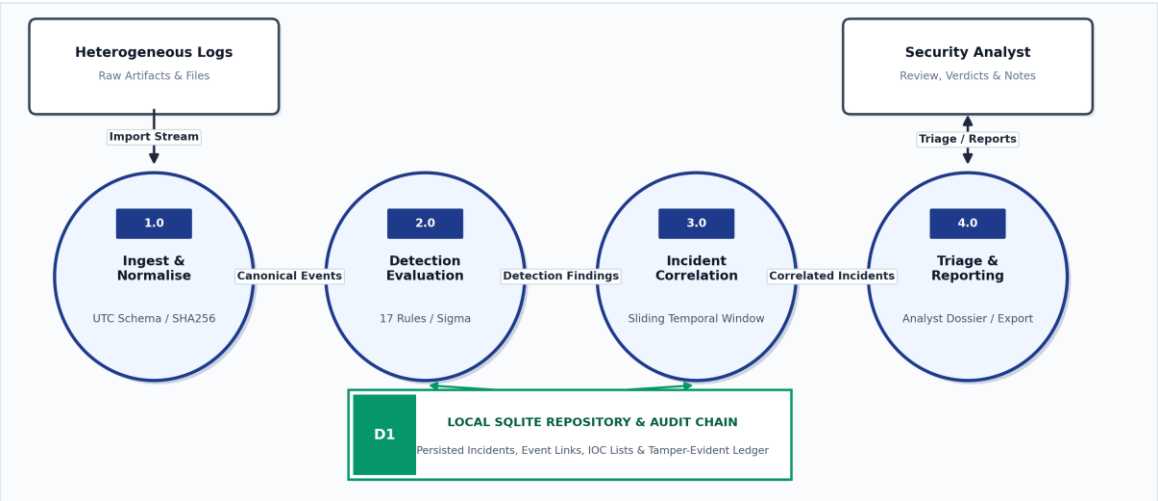


Figure 4.2: Project Data Flow Diagram

4.4.2 Incident Correlation Workflow

Correlation uses shared actor entities and a configured time window to link findings. Target or victim entities are not used as general linking keys, which helps reduce unrelated findings being merged. Severity and confidence are derived from rule results and corroborating evidence; the analyst should review the contributing factors.

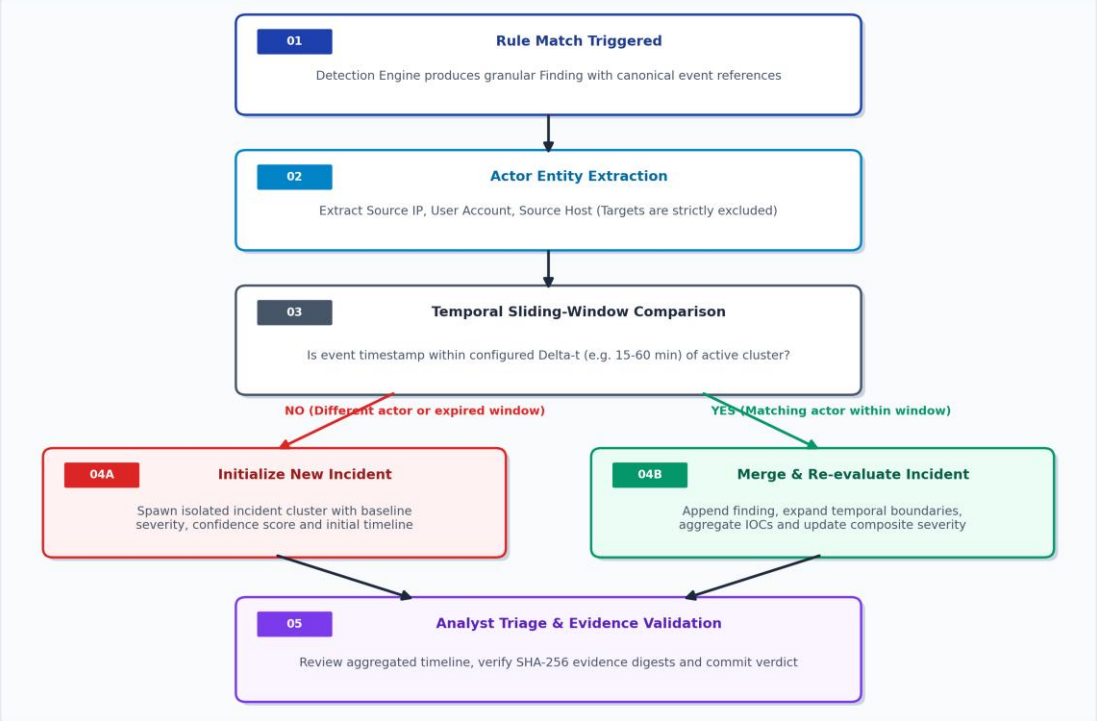


Figure 4.3: Incident Correlation Workflow

4.5 Detection and Correlation Design

The project documentation describes 17 built-in detection rules covering behaviours such as port scanning, web scanning, brute force, password spraying, MFA fatigue, suspicious command lines, persistence, lateral movement, beaconing, large outbound transfers and IOC matches. Rule results are investigative leads. Their accuracy depends on log quality, rule thresholds and the environment being analysed.

4.6 Security Considerations

- Validate file uploads and enforce parsing and size limits.
- Use safer archive handling and reject unsafe XML constructs such as DOCTYPE and ENTITY declarations.
- Validate browser origins and apply security headers.
- Support access tokens for non-localhost binding and Docker use.
- Retain evidence hashes and verify integrity before relying on evidence.
- Use hash-chained audit records while protecting the underlying data directory.
- Encrypt exported case files using the documented passphrase-based mechanism.
- Review generated reports for sensitive information before sharing.

The project is designed for local, single-analyst use. The encrypted case-file export does not mean that the live SQLite database is encrypted. The application should not be exposed as an unrestricted multi-user service.

4.7 Test Case Design

Table 4.2: Test Case Design

Test ID	Description	Expected Output
TC01	Import a supported structured log file.	Events are parsed and normalised.
TC02	Import malformed or unsupported input.	Invalid records are reported without silent acceptance.
TC03	Verify evidence file hash.	Integrity status matches the recorded SHA-256 value.
TC04	Run a built-in detection rule test.	Expected test event produces a finding.
TC05	Load a supported Sigma rule.	Supported rule is parsed and evaluated.
TC06	Correlate findings sharing actor entities.	Related findings are grouped within configured time window.
TC07	Review an incident timeline.	Linked events appear in chronological order.
TC08	Add an analyst note or verdict.	Case state is persisted.
TC09	Export an incident report.	Requested report format is generated.
TC10	Run automated test suite.	Test summary is produced without failed tests in the recorded run.

Chapter 5

IMPLEMENTATION AND TESTING

5.1 Implementation Approach

5.1.1 Development Environment

The project is implemented in Python and uses a local browser-based interface. The repository includes application modules, web assets, documentation and automated tests. The declared project configuration identifies Python 3.11 or newer as the runtime requirement. Dependencies are recorded in the project dependency files.

5.1.2 Modular Development

The implementation separates ingestion, format handling, detection, Sigma processing, correlation, evidence management, persistence, API security, reporting and export responsibilities. This separation makes the application easier to test and allows individual components to be improved without placing all logic in one module.

5.1.3 Interface Integration

The local web console presents the analyst with dashboard and investigation views. It provides access to findings, incidents, event timelines, evidence context, IOCs and report-generation functions. The command-line workflow provides an additional way to run supported analysis tasks.

5.2 Log Ingestion and Normalisation

The ingestion layer accepts supported structured and text-based log formats, including CSV, JSON, NDJSON and selected source-specific formats documented by the project. Recognised source fields are mapped to a canonical event model. Timestamps are normalised to UTC. Records without a parseable timestamp are rejected and counted rather than being assigned an invented time. The application exposes data-quality information so that the analyst can understand the limitations of the imported dataset.

5.3 Detection and Correlation Implementation

The detection engine evaluates canonical events using built-in rules and supported imported Sigma rules. When a rule matches, a finding is created with references to the relevant event records. The correlation engine then examines shared actor entities and time windows to group related findings into incidents. The interface exposes severity and confidence context for review. Detection does not automatically establish malicious intent.

5.4 Security Implementation

Evidence handling uses SHA-256 hashes to support integrity verification. The application includes upload and parsing limits, safer archive processing, XML protections, browser-origin validation, security headers and access-token support for selected non-local deployment modes. Case-file export can use passphrase-based encryption. Audit records use hash chaining to make

certain changes detectable. These controls reduce selected risks but do not make the application suitable for unrestricted enterprise deployment.

5.5 Testing Approach

The supplied project includes an automated Pytest suite covering ingestion, supported formats, detection, correlation, API behaviour, security controls, hardening, evidence and audit handling, persistence, reporting, redaction, Sigma processing and export behaviour.

Table 5.1: Automated Test Result

Test Metric	Recorded Result
Tests passed	100% (Batch 1: 62%, Batch 2: 100% - all tests passed)
Tests failed	0 (Zero test failures)
Warnings	1222 FastAPI/asyncio deprecation warnings (Python 3.14 upstream) + 1 Sigma regex warning
Execution time	29.81 seconds (Executed via python -m pytest -q in PowerShell)

The recorded command was: python -m pytest -q. The latest captured execution completed with 175 passed, 2 failed and 71 warnings. The failing tests were test_nested_json_and_ndjson and test_clear_marking_removes_identifiers_from_every_report_format. The first failure indicates a mismatch between the expected username and the anonymised test fixture. The second requires review of the report-redaction assertion and exported content to determine whether an identifier is genuinely exposed or the assertion is matching an incidental string. Warnings included a Starlette/httpx test-client deprecation notice, a pandas weekday accessor deprecation notice, and a Sigma regular-expression capturing-group warning. These warnings are distinct from the two test failures. The test suite must be rerun after corrections before claiming a clean pass.

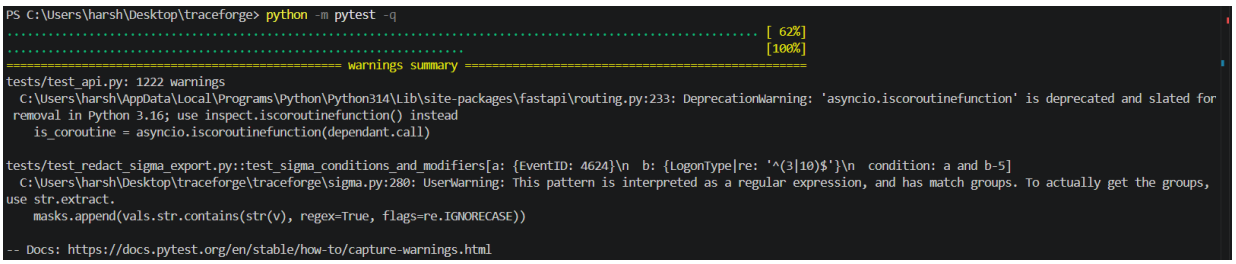


Figure 5.1: Automated Test Execution

5.6 Challenges and Improvements

The main implementation challenge is handling different log formats while preserving the meaning and provenance of the original records. Timestamp normalisation, field mapping and malformed input handling must be performed carefully. Another challenge is correlating findings without merging unrelated activity. The project addresses this by using shared actor entities and time windows and by exposing confidence factors for analyst review. Further improvements can focus on broader source support, richer validation datasets and stronger multi-user deployment controls.

Chapter 6

RESULTS AND OUTPUT

6.1 Final Output

The completed project provides a local workbench for importing supported security logs, normalising events, generating detection findings, correlating findings into incidents and reviewing the supporting evidence. The analyst can inspect timelines, add notes and indicators, record verdicts and generate reports. The application processes user-supplied files and does not provide continuous monitoring of endpoints or networks.

6.2 Analyst Workflow

1. Launch the TraceForge application and open the local console.
2. Select supported log files for analysis.
3. Review the import summary, accepted records and data-quality warnings.
4. Open the findings view and inspect rule matches and supporting events.
5. Review correlated incidents and inspect their event timelines.
6. Validate the evidence and record analyst notes, verdicts or IOCs.
7. Generate an incident report or export the required data format.

6.3 Project Screenshots

The following screenshot positions should be replaced with images captured from the final project on the presentation computer. Each screenshot should be cropped to the application window, remain readable and include a short explanation. Use synthetic or sanitised logs. Do not expose real usernames, tokens, email addresses, internal hostnames or confidential IP information.

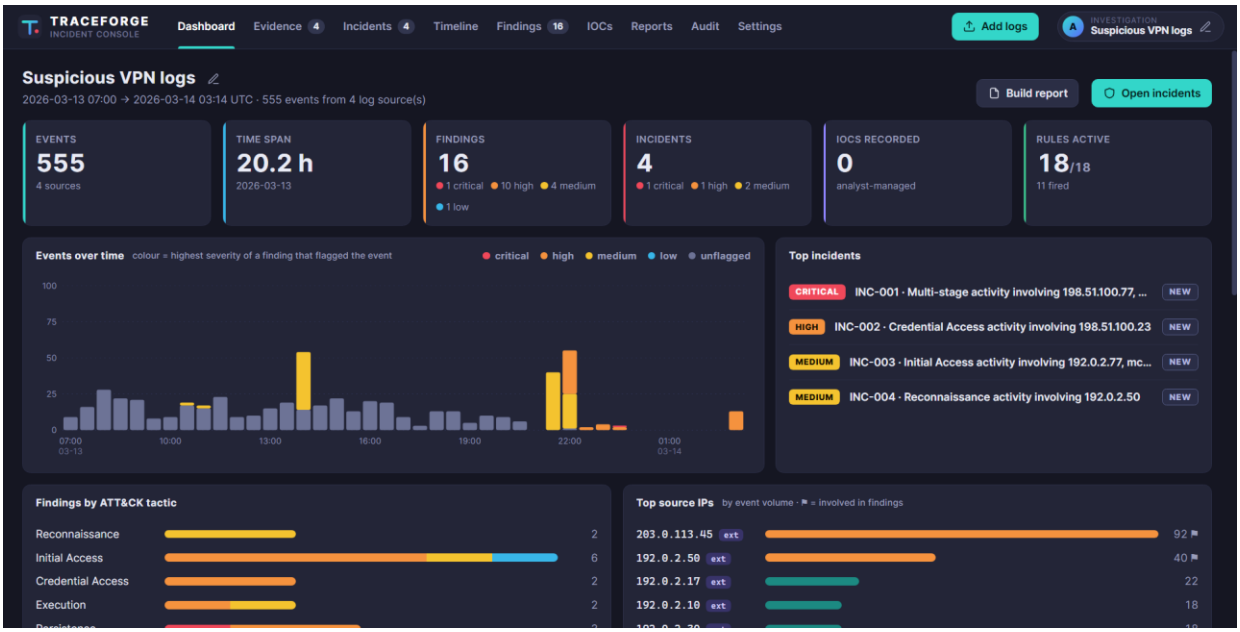


Figure 6.1: TraceForge Dashboard

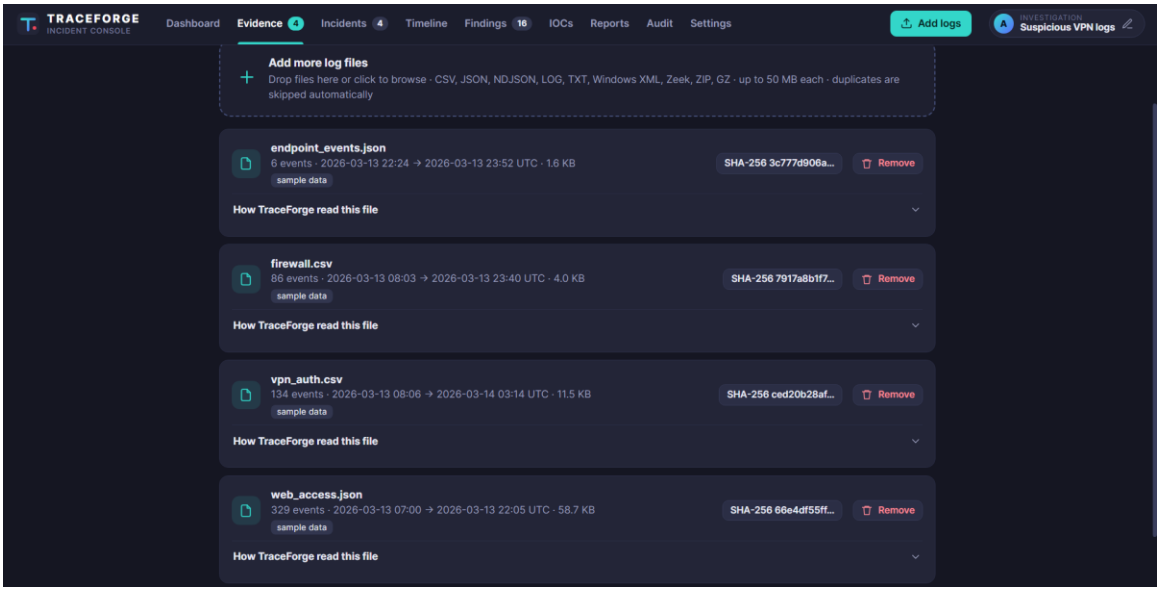


Figure 6.2: Evidence Import and Data Quality

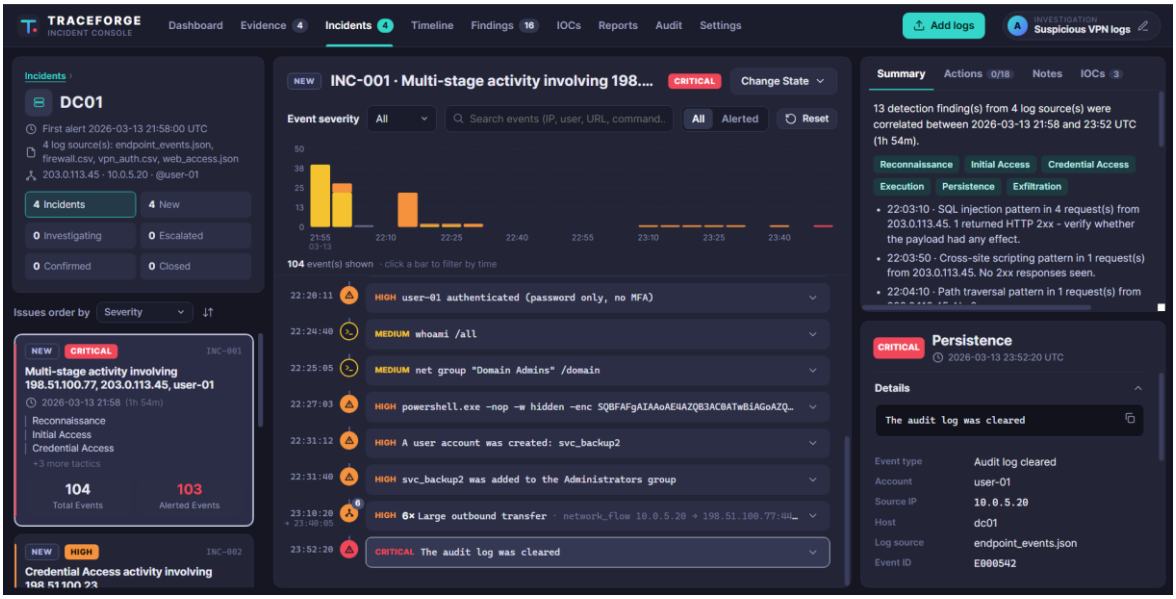


Figure 6.3: Detection Findings and Incident Timeline

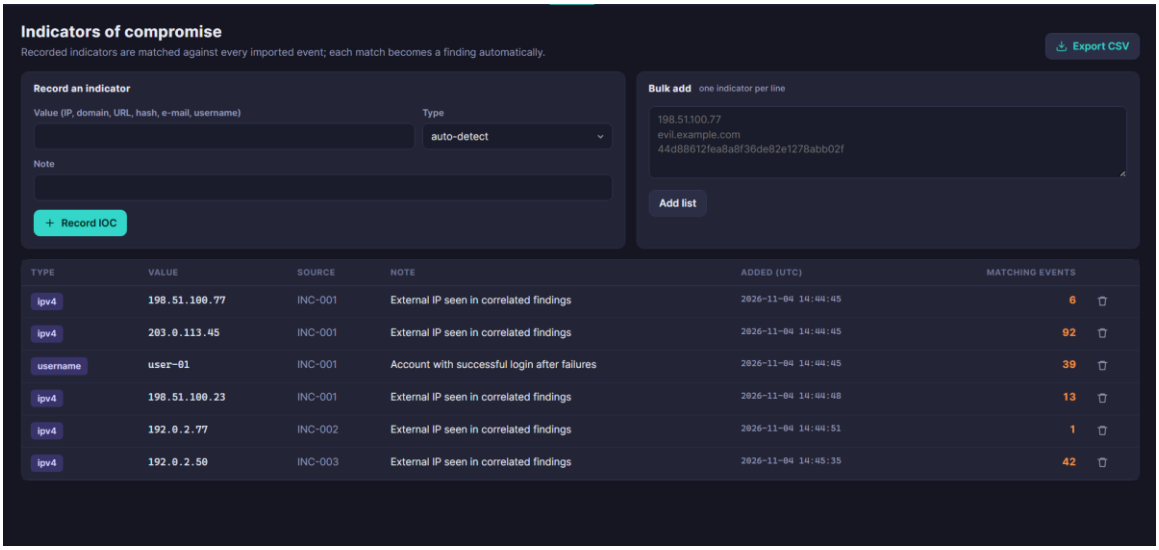


Figure 6.4: IOC Management

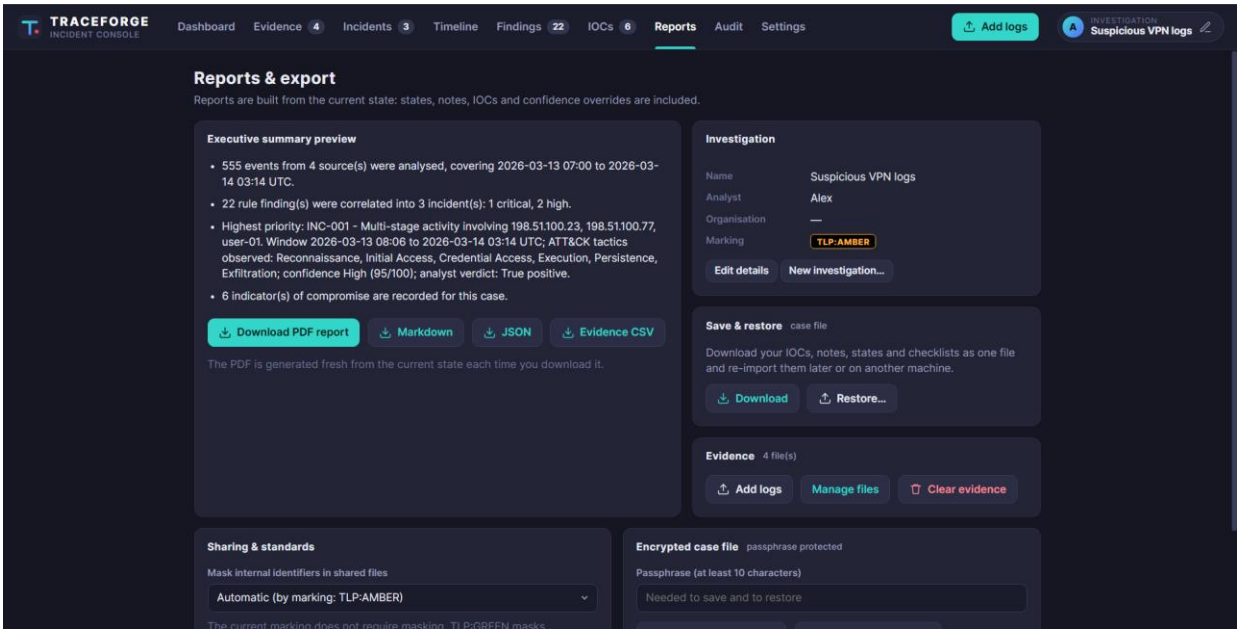


Figure 6.5: Reports & Investigation Summary

6.4 Results Summary

Table 6.1: Results Summary

Project Requirement	Recorded Result
Log ingestion	Project supports documented log formats and reports parsing outcomes.
Event normalisation	Recognised fields are mapped into a canonical event structure and timestamps normalised to UTC.
Detection	Built-in rules and supported Sigma rules generate findings.
Incident correlation	Related findings are grouped using actor entities and time windows.
Evidence integrity	SHA-256 hashes are recorded and can be verified.
Analyst workflow	Incidents, timelines, notes, verdicts and IOCs can be reviewed.
Reporting	PDF, Markdown, JSON and CSV outputs are documented, with selected STIX and ATT&CK exports.
Automated tests	175 passed, 2 failed, 71 warnings in the latest captured run.
Sample demonstration	555 events from four sources, 17 findings and four correlated incidents.

6.5 Limitations

- The application analyses user-supplied files and does not perform continuous live monitoring.
- The synthetic demonstration data is not evidence of a real security incident.
- Detection rules may generate false positives or miss behaviours that are not represented in the available logs or rules.
- The project is designed for local, single-analyst use and does not provide full multi-user identity and role management.
- The encrypted case-file export does not encrypt the active SQLite database.
- The audit hash chain alone cannot prove that the latest audit records have not been truncated.
- Native binary EVTX parsing and external threat-intelligence lookups are not included in the documented current scope.
- The tool does not automatically contain threats or remediate affected systems.

Chapter 7

FUTURE SCOPE

7.1 Broader Log Source Support

Future versions can add native binary EVTX parsing and additional endpoint, identity, cloud and network log sources. Each parser should include representative test data and clear data-quality reporting.

7.2 Improved Detection Engineering

Detection coverage can be expanded through more validated rules, rule testing, configurable thresholds and analyst feedback. A rule-validation workflow can help identify false positives and document the expected input fields.

7.3 Advanced Incident Visualisation

An attack-graph view could show relationships between hosts, users, IP addresses, findings and incidents. This may help analysts understand how events are connected across multiple sources.

7.4 Threat Intelligence Integration

Approved threat-intelligence services could be integrated to enrich indicators. Such integration should include privacy controls, API-key protection, rate limits and clear indication of the source and age of intelligence.

7.5 Multi-User Case Management

A future team version could introduce authentication, role-based access control, case assignment, approvals and case-level permissions. A secure deployment would require TLS, managed secrets, stronger identity controls and a suitable database.

7.6 Performance and Evaluation

The application can be benchmarked against larger representative datasets. Evaluation should measure ingestion speed, memory use, detection precision, false-positive rates and correlation quality. Results should be reproducible and should distinguish synthetic tests from real-world operational evaluation.

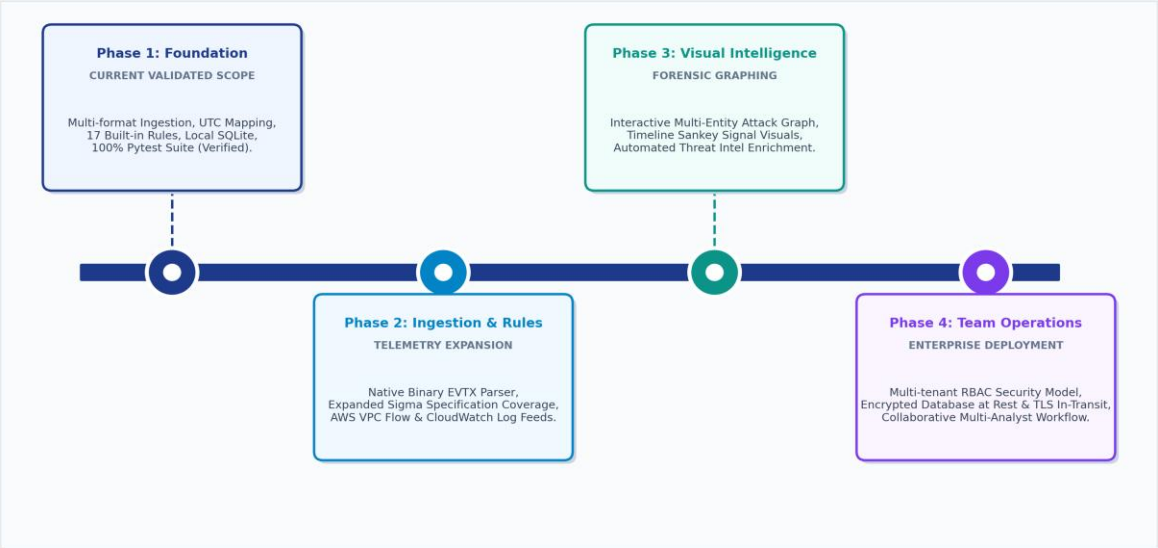


Figure 7.1: Recommended Future Development Roadmap

Chapter 8

CONCLUSION

8.1 Conclusion

TraceForge demonstrates an integrated local workflow for analysing security logs from multiple sources, normalising events, identifying potentially suspicious patterns, correlating findings into incidents and producing evidence-linked reports. It brings ingestion, detection, correlation, evidence integrity and analyst triage into one workbench. The application provides event timelines, IOC tracking, notes, verdicts, data-quality information and selected report exports.

The supplied project archive includes a synthetic demonstration involving 555 events from four sources, 17 findings and four correlated incidents. The latest recorded automated test execution during report revision produced 175 passed tests, 2 failed tests and 71 warnings. This is not a fully passing test result. The failures and warnings should be resolved or documented, followed by a complete rerun, before the project is presented as fully validated. These results do not establish detection effectiveness against every real-world attack.

TraceForge should therefore be considered an analyst-support and educational project rather than an autonomous incident-response platform or a replacement for enterprise security monitoring. Its limitations provide a clear direction for future improvements in parser coverage, detection validation, deployment security and team-based case management.

8.2 Learning Outcomes

- Understanding how security logs from different sources can be normalised.
- Learning how rule-based detections can identify suspicious event patterns.
- Understanding IOC matching and event correlation using shared entities and time windows.
- Applying SHA-256 hashing to evidence integrity verification.
- Working with local case persistence and structured incident records.
- Understanding the importance of analyst validation, false positives and evidence provenance.
- Preparing automated tests, technical documentation and investigation reports.

BIBLIOGRAPHY

[1] TraceForge project source code and documentation supplied with the project archive, including README.md, docs/ARCHITECTURE.md, docs/USER_GUIDE.md, docs/DETECTION_RULES.md, docs/DEPLOYMENT.md and THREAT_MODEL.md.

[2] MITRE, “MITRE ATT&CK,” https://attack.mitre.org/ (accessed [insert date]).

[3] SigmaHQ, “Sigma Generic Signature Format for SIEM Systems,” https://sigmahq.io/ (accessed [insert date]).

[4] Python Software Foundation, “Python Documentation,” https://docs.python.org/3/ (accessed [insert date]).

[5] FastAPI, “FastAPI Documentation,” https://fastapi.tiangolo.com/ (accessed [insert date]).

[6] Pytest, “Pytest Documentation,” https://docs.pytest.org/ (accessed [insert date]).

[7] OASIS, “STIX Version 2.1,” https://docs.oasis-open.org/cti/stix/v2.1/ (accessed [insert date]).

[8] FIRST, “Traffic Light Protocol (TLP) 2.0,” https://www.first.org/tlp/ (accessed [insert date]).

[9] OWASP Foundation, “OWASP Cheat Sheet Series,” https://cheatsheetseries.owasp.org/ (accessed [insert date]).